IBM Z DATATHON 2026 · REAL-TIME AI FOR CRITICAL DECISIONS · SSF2026

LegacyLens

Prove what legacy COBOL decides — then add real-time AI you can trust.

IBM LinuxONE · z17 | z3 proofs | ONNX on the AI accelerator | IBM Granite | DMN 1.3

Team Aviator

Piyush Anand · Somesh Srivastava · Praman Patel · Paraj Prasad · Mohit Kumar A

Aviator_IBM Z Datathon 2026 Project · github.com/GeekSomesh/LegacyLens

01 · THE PROBLEM

Critical decisions still run on COBOL that nobody can fully explain

Banks, insurers and public agencies decide credit limits, fraud holds and account updates in COBOL written decades ago. The rules live only in the code — and rewriting it is slow, expensive and risky.

Opaque rules

The authors have retired and the documentation is missing or wrong. No one can say exactly what the program decides.

Risky change

A one-line edit can silently change outcomes for thousands of customers, and nobody sees it before release.

Hidden defects

Lost signs, digits dropped on overflow and wrong status codes survive for years without a single error message.

AI without trust

A model bolted onto unexplained code makes decisions no one can justify to a customer, an auditor or a regulator.

Even IBM's own sample COBOL hides it.
Replace a balance with `-123.45` and the program stores `+123.45`: a debit becomes a credit. Add to a large balance and ten million silently disappears.

Found by LegacyLens with the z3 solver and confirmed on the compiled program (IBM Z Open Editor sample SAM2, Apache-2.0).

Regulators expect decisions that are **explainable and auditable**. Legacy logic is a black box — and adding AI makes it darker.

02 · OUR SOLUTION

LegacyLens: prove it, change it safely, add AI you can trust

From COBOL source to verified rules, safe changes and real-time hybrid decisions — on one IBM LinuxONE server.

1

Extract

Parse COBOL byte for byte — copybooks, REDEFINES, COMP-3, OCCURS — and turn every decision path into a readable rule.

2

Prove

The z3 solver removes impossible paths, simplifies each rule, gives an example input and finds overflow, sign loss and division by zero.

3

Verify

Every rule is checked against the compiled program, byte for byte, on tens of thousands of inputs.

4

Change safely

z3 finds every input region where a new version decides differently; both compiled programs confirm it before release.

5

Decide in real time

An ONNX risk model on the z17 AI accelerator can hold or escalate a COBOL decision — never approve. IBM Granite explains.

Governance built in: **DMN 1.3 export** · **hash-chained review audit trail** · **Granite rule documentation** · **fault injection** — and no data ever leaves the IBM LinuxONE server.

03 · RESULTS AT A GLANCE

Measured on the IBM z17, not promised

100%

rules = compiled COBOL on 63,007 records and calls, 300 whole-program file runs and 10,000 construct tests

109 → 17

IBM SAM2 code paths reduced to proven rules; 92 paths proved impossible by z3

14

findings in four programs, each with an input that reproduces it on the compiled code

6 / 6

change-impact regions found by z3, all confirmed by both compiled programs

57%

of defaulters kept from a limit increase by the hybrid AI on 4,501 held-out accounts

0

planted bugs missed: 91 behaviour-changing mutants caught by tests or z3

15 µs

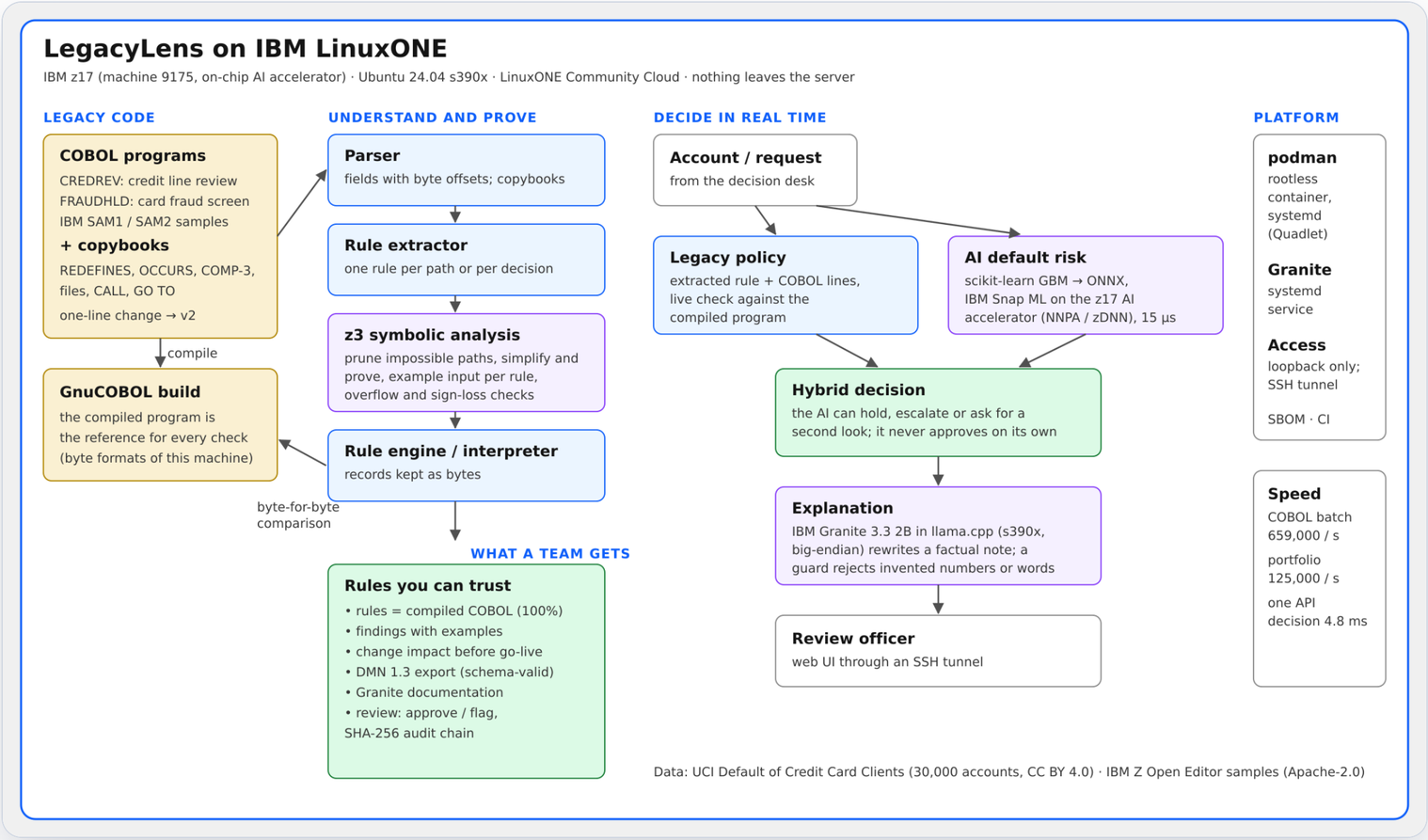
ONNX risk score for one account on the z17 on-chip AI accelerator

125,000/s

hybrid decisions per second for the whole 30,000-account portfolio

Every figure is produced by the code on the LinuxONE server and stored in `results/*.json` in the repository.

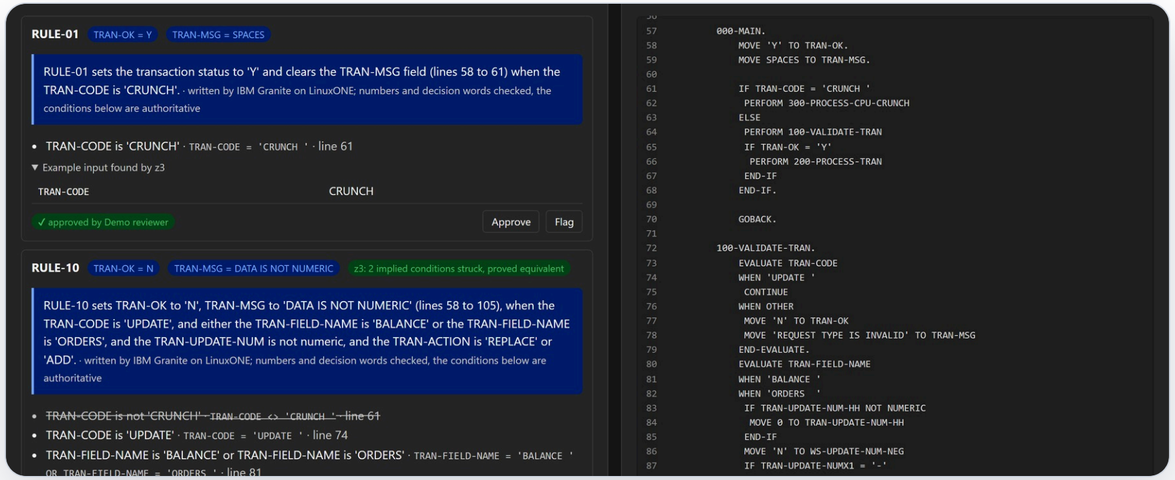
One LinuxONE server: legacy code, proofs and AI side by side



- **Legacy code.** COBOL programs and copybooks; the GnuCOBOL build on s390x is the reference for every check.
- **Understand and prove.** Parser with byte offsets, rule extractor, z3 symbolic analysis, rule engine.
- **Decide in real time.** Legacy policy + AI default risk → hybrid decision → Granite explanation for the review officer.
- **Platform.** Rootless podman container under systemd, IBM Granite as a systemd service; only SSH is exposed.

Reading COBOL the way the machine does

- **Byte-exact storage model:** REDEFINES overlays, OCCURS tables, COMP-3 packed decimal, big-endian COMP, zoned signs, edited pictures.
- **Real programs:** COPY … REPLACING, sequential and variable-length files, ACCEPT/DISPLAY exactly as GnuCOBOL does them.
- **Control flow:** IF/EVALUATE, PERFORM THRU, GO TO, inline and VARYING loops, CALL to subprograms.
- **Two rule modes:** whole-program paths (CREDREV, FRAUDHLD, SAM2) or one rule per decision statement (SAM1).
- **Every rule shows** the decision, its conditions in plain English and the exact COBOL lines.
- **Never guesses:** anything outside the supported subset is reported.

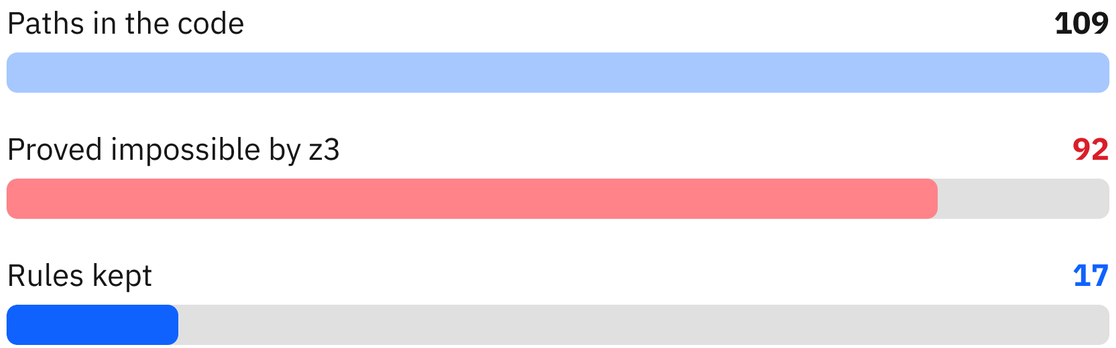


IBM SAM2 rule on the live system: COBOL lines on the right, a z3 example input, Granite documentation and the review status.

06 · STEP 2 — PROVE

z3 turns 109 code paths into 17 proven rules

IBM SAM2 · CUSTOMER UPDATE SUBROUTINE



44
redundant conditions removed

all
simplifications proved equivalent

none
overlapping rules

WHAT Z3 DOES FOR EVERY RULE

- Proves which paths can never run and drops them.
- Strikes conditions implied by the others — and proves the simplified rule equals the original.
- Finds a concrete example input that reaches the rule.
- Checks that no two rules can fire for the same input.
- Finds value risks with an example: **silent overflow**, **sign loss**, **division by zero**.
- Unrolls counted loops and reads constant tables (the FRAUDHLD high-risk country table).

> Whole SAM2 analysis on the z17: **6.1 s**.

07 · STEP 3 — VERIFY

Every rule is checked against the compiled program

PROGRAM	WHAT IS COMPARED	INPUTS	AGREEMENT
CREDREV credit line review	6 rules vs the compiled program's output line, byte for byte	35,260 records	**100%**
FRAUDHLD card fraud screen	8 rules, risk-country table built with REDEFINES and OCCURS	20,212 requests	**100%**
IBM SAM2 (zopeneditor sample)	17 rules vs CALLs to the compiled subroutine, all 4 parameters byte for byte	7,535 calls	**100%**
IBM SAM1 batch + CALL SAM2	whole program: output file, report, console and return code	300 file runs	**100%**
Construct suite (5 programs)	GO TO, PERFORM THRU, OCCURS, COMP-3, edited pictures, 88-levels, INITIALIZE	10,000 records	**100%**
DMN 1.3 export	exported decision model vs the compiled COBOL (CREDREV · FRAUDHLD)	35,262 · 25,192	**100%**

Reference: GnuCOBOL 3.1.2 builds on s390x. CREDREV inputs are 30,000 real accounts plus boundary and random records. One SAM1 run with undefined COBOL behaviour (arithmetic on text) is reported as a finding, not counted.

Real defects, each with an input that proves it

WHERE	KIND	WHAT HAPPENS (CONFIRMED ON THE COMPILED PROGRAM)
IBM SAM2 · line 121	**sign lost**	**Replace a balance with −123.45 → the program stores +123.45: a debit becomes a credit**
IBM SAM2 · line 124	**silent overflow**	**Balance 9,999,999.99 + update 999,999.99 → computed 10,999,999.98, stored 999,999.98: ten million lost**
IBM SAM2 · 124–134	sign / overflow	The same two defects on CUST-ACCT-BALANCE (ADD) and CUST-ORDERS-YTD — 5 more findings
IBM SAM1	status mismatch ×2	710-READ-TRAN-FILE and 740-WRITE-CUSTOUT-FILE test one file's status but report CUSTOMER-FILE's
IBM SAM1 · line 357	file never closed	CUSTOMER-FILE-OUT is opened but never closed
IBM SAM1 · line 329	outside a table	WS-CUST-MONTH(13) on a 12-entry table in 142 of 301 runs: bytes of the next field are used
IBM SAM1 → SAM2	undefined	An UPDATE for a contact record reaches SAM2, which treats it as a customer record (1 of 301 runs)
CREDREV · line 46	silent overflow	Utilisation of 100,000% (limit 2, bill 2,000) is stored as 0
FRAUDHLD · line 68	silent overflow	WS-RATIO drops its high-order digits (amount 100,000 against a 30-day average of 1)

09 · CHANGE IMPACT

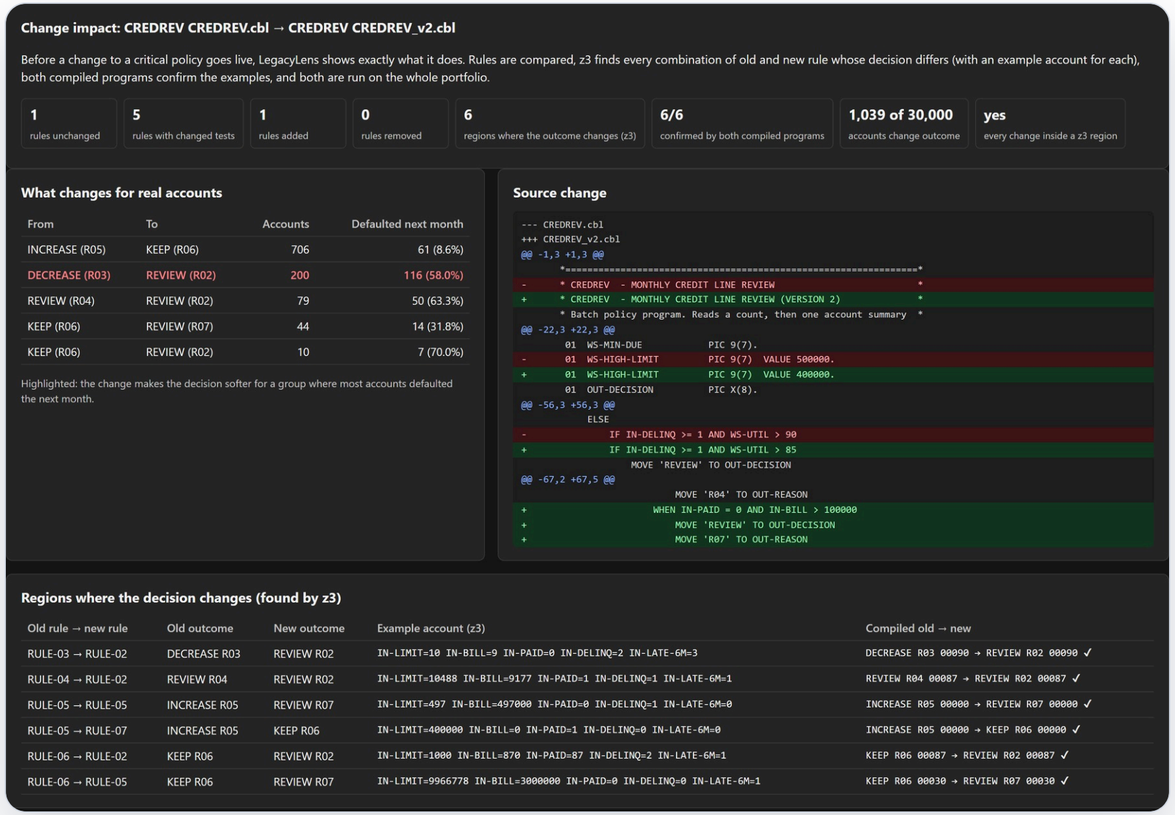
See what a change does — before it ships

CREDREV v1 → v2 looks like a simple tightening:

- utilisation threshold for late accounts 90% → 85%
- high-limit cap 500,000 → 400,000
- new rule R07: no payment and a bill over 100,000 → REVIEW

6 regions found by z3 | 6/6 confirmed by both compiled programs | 1,039 of 30,000 accounts change | every change inside a z3 region

FROM	TO	ACCOUNTS	DEFAULTED
INCREASE (R05)	KEEP (R06)	706	8.6%
DECREASE (R03)	**REVIEW (R02)**	**200**	**58.0%**
REVIEW (R04)	REVIEW (R02)	79	63.3%
KEEP (R06)	REVIEW (R07 · R02)	54	38.9%



Change impact: CREDREV CREDREV.cbl → CREDREV CREDREV_v2.cbl

Before a change to a critical policy goes live, LegacyLens shows exactly what it does. Rules are compared, z3 finds every combination of old and new rule whose decision differs (with an example account for each), both compiled programs confirm the examples, and both are run on the whole portfolio.

1	5	1	0	6	6/6	1,039 of 30,000	yes
rules unchanged	rules with changed tests	rules added	rules removed	regions where the outcome changes (z3)	confirmed by both compiled programs	accounts change outcome	every change inside a z3 region

What changes for real accounts

From	To	Accounts	Defaulted next month
INCREASE (R05)	KEEP (R06)	706	61 (8.6%)
DECREASE (R03)	REVIEW (R02)	200	116 (58.0%)
REVIEW (R04)	REVIEW (R02)	79	50 (63.3%)
KEEP (R06)	REVIEW (R07)	44	14 (31.8%)
KEEP (R06)	REVIEW (R02)	10	7 (70.0%)

Highlighted: the change makes the decision softer for a group where most accounts defaulted the next month.

Source change

```
--- CREDREV.cbl
+++ CREDREV_v2.cbl
@@ -1,3 +1,3 @@
      *============================================================*
-     * CREDREV  - MONTHLY CREDIT LINE REVIEW                      *
+     * CREDREV  - MONTHLY CREDIT LINE REVIEW (VERSION 2)          *
      * Batch policy program. Reads a count, then one account summary  *
@@ -22,3 +22,3 @@
       01  WS-MIN-DUE          PIC 9(7).
-      01  WS-HIGH-LIMIT       PIC 9(7)  VALUE 500000.
+      01  WS-HIGH-LIMIT       PIC 9(7)  VALUE 400000.
       01  OUT-DECISION        PIC X(8).
@@ -56,3 +56,5 @@
           ELSE
-              IF IN-DELINQ >= 1 AND WS-UTIL > 90
+              IF IN-DELINQ >= 1 AND WS-UTIL > 85
                   MOVE 'REVIEW' TO OUT-DECISION
@@ -67,2 +67,5 @@
                       MOVE 'R04' TO OUT-REASON
+                  WHEN IN-PAID = 0 AND IN-BILL > 100000
+                      MOVE 'REVIEW' TO OUT-DECISION
+                      MOVE 'R07' TO OUT-REASON
```

Regions where the decision changes (found by z3)

Old rule → new rule	Old outcome	New outcome	Example account (z3)	Compiled old → new
RULE-03 → RULE-02	DECREASE R03	REVIEW R02	IN-LIMIT=10 IN-BILL=9 IN-PAID=0 IN-DELINQ=2 IN-LATE-6M=3	DECREASE R03 00090 → REVIEW R02 00090 ✓
RULE-04 → RULE-02	REVIEW R04	REVIEW R02	IN-LIMIT=10488 IN-BILL=9177 IN-PAID=1 IN-DELINQ=1 IN-LATE-6M=1	REVIEW R04 00087 → REVIEW R02 00087 ✓
RULE-05 → RULE-05	INCREASE R05	REVIEW R07	IN-LIMIT=497 IN-BILL=497000 IN-PAID=0 IN-DELINQ=1 IN-LATE-6M=0	INCREASE R05 00000 → REVIEW R07 00000 ✓
RULE-05 → RULE-07	INCREASE R05	KEEP R06	IN-LIMIT=400000 IN-BILL=0 IN-PAID=1 IN-DELINQ=0 IN-LATE-6M=0	INCREASE R05 00000 → KEEP R06 00000 ✓
RULE-06 → RULE-02	KEEP R06	REVIEW R02	IN-LIMIT=1000 IN-BILL=870 IN-PAID=87 IN-DELINQ=2 IN-LATE-6M=1	KEEP R06 00087 → REVIEW R02 00087 ✓
RULE-06 → RULE-05	KEEP R06	REVIEW R07	IN-LIMIT=9966778 IN-BILL=3000000 IN-PAID=0 IN-DELINQ=0 IN-LATE-6M=1	KEEP R06 00030 → REVIEW R07 00030 ✓

The surprise: **200 accounts move from DECREASE to a softer REVIEW — and 58% of them defaulted the next month.**

Legacy policy + AI risk, decided in milliseconds

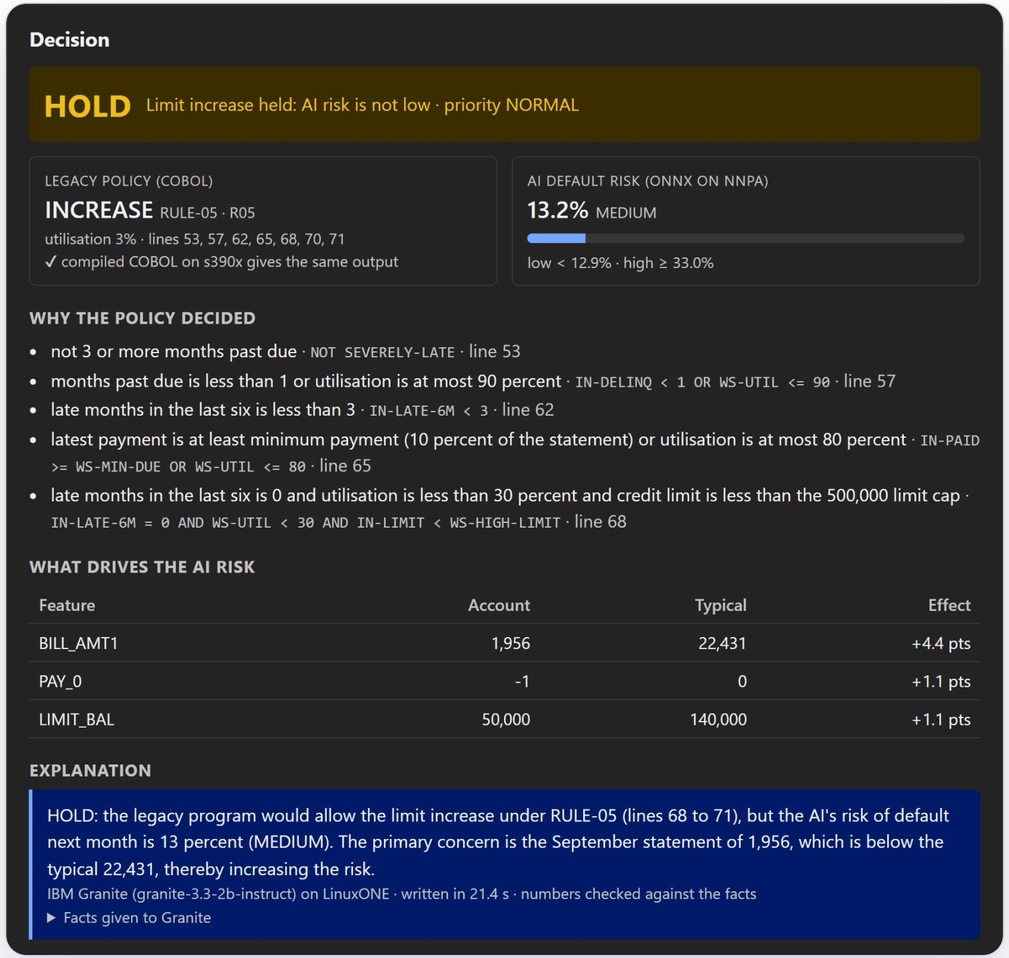
1 COBOL policy
The extracted rules decide and name the COBOL lines; the compiled program checks the same record live.

2 AI default risk
Gradient-boosted model exported to ONNX, served by IBM Snap ML on the z17 AI accelerator: probability, band, top factors.

3 Hybrid rule
The AI may **hold** an increase or **escalate** a review — it can never approve on its own.

4 Explanation
IBM Granite rewrites the facts in plain English; a guard rejects any new number, percentage or decision word.

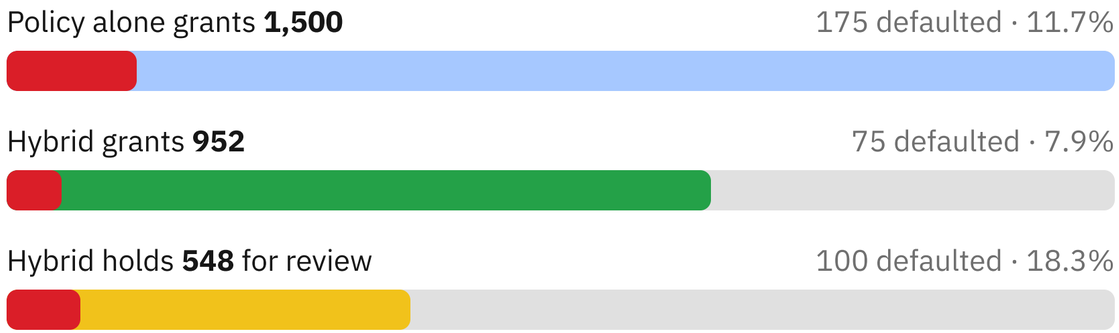
5 Review officer
Decision, reasons, lines and explanation on one screen — 4.76 ms per account through the web service (p50).



Account 4750 on the live system: COBOL says **INCREASE** (RULE-05), AI risk is **13.2% · MEDIUM**, so the increase is **HELD** for review.

The AI holds risky increases — and never approves on its own

LIMIT INCREASES · 4,501 HELD-OUT TEST ACCOUNTS



57% of the defaulters who would have received a limit increase are stopped for review.

REVIEW QUEUE ORDERED BY AI RISK

PRIORITY	ACCOUNTS	DEFAULTED
URGENT	219	**54.8%**
NORMAL	537	15.5%
LOW	232	15.1%

Risk model

Gradient-boosted trees (scikit-learn) → ONNX (112 KB), trained and tested on LinuxONE.

ROC AUC	**0.777**
Average precision	**0.547**
High band: precision / recall	**57.2% / 52.1%**
Test default rate	22.1%

Same ONNX file, three runtimes

ONNX reference evaluator, Snap ML on the AI accelerator and Snap ML on the CPU give the same risk band for **100%** of accounts.

Data: UCI Default of Credit Card Clients (Taiwan, 2005, CC BY 4.0). It demonstrates the pipeline; it is not a credit model to deploy.

12 · GOVERNANCE

Built for audit: exportable, reviewable, tested

DMN 1.3 export

Rules as standard decision tables (FEEL, FIRST hit policy), validated against the OMG schema. The exported model gives the same decisions as the compiled COBOL on **35,262 / 35,262** (CREDREV) and **25,192 / 25,192** (FRAUDHLD) records.

Rule review with an audit trail

Approve or flag each rule. Every action is appended to a **SHA-256 hash-chained** log; tampering is detected, and an approval goes stale when the rule changes.

Rule documentation by IBM Granite

Every rule of all four programs documented in plain English by Granite 3.3 on LinuxONE. A guard checks numbers, percentages, decision words and rule ids: **57 of 59** accepted; the rest fall back to the factual text.

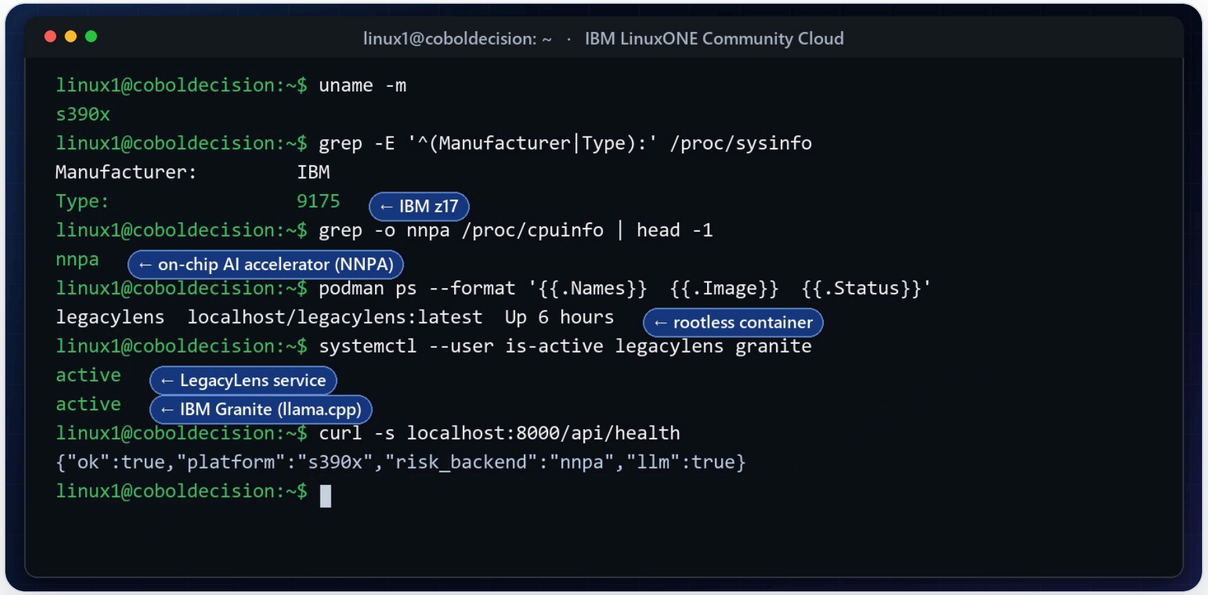
Fault injection

PROGRAM	MUTANTS	CAUGHT	HARMLESS (Z3)	MISSED
CREDREV	80 (74 compile)	70 + 2 by z3	2	**0**
IBM SAM2	50 (37 compile)	19	18	**0**

19 automated tests run on LinuxONE and on GitHub Actions (x86).

Everything runs on an IBM z17

- **IBM z17** (machine type 9175) virtual server in the LinuxONE Community Cloud, Ubuntu 24.04 on s390x.
- **On-chip AI accelerator:** ONNX model served by IBM Snap ML through zDNN (`zdnn_tensors`).
- **COBOL on s390x:** GnuCOBOL builds are the reference; native big-endian binary, packed and zoned decimal.
- **Proofs on the box:** z3 runs on the same machine.
- **Generative AI on the box:** IBM Granite 3.3 2B in llama.cpp built for the s390x vector extensions, big-endian weights — no external API.
- **Operations:** rootless podman container under systemd; only SSH is exposed.



Real output on the server: s390x, machine type 9175 (IBM z17), the NNPA facility, the running container and both services.

Real-time at portfolio scale

124,957/s
hybrid decisions for the whole 30,000-account portfolio: compiled COBOL + ONNX risk + hybrid rule

658,779/s
compiled COBOL policy, batch of 30,000 accounts

15.1 µs
ONNX risk score for one account on the AI accelerator (p50; p99 20.2 µs)

4.76 ms
one account through the web service with a live COBOL check (p50; p95 6.9 ms)

SAME ONNX MODEL, AI ACCELERATOR VS CPU (SNAP ML, 30,000 ACCOUNTS)

TREE MODEL	AI ACCELERATOR (NNPA)	CPU (Z17 VECTOR)
200 trees, depth 3 (deployed)	172,549 / s	947,986 / s
500 trees, depth 6	39,553 / s	175,607 / s
1,000 trees, depth 8	15,063 / s	62,696 / s

An honest result: on this shared virtual server, Snap ML's CPU path with z17 vector instructions is faster than the NNPA path for gradient-boosted trees of every size we tried. Both give **identical risk bands**. The accelerator path is built, measured and ready for larger and deep-learning models, where it is designed to pay off.

Who benefits — and how

Lower risk

Every rule verified; hidden defects exposed with an input that reproduces them.

Safer change

See exactly which customers a policy change affects — before release.

Trusted AI

The AI only adds caution, never approves alone, and every decision is explained.

No rewrite

Standard DMN export and an audit trail — and the data never leaves IBM Z.

Banks, insurers, public agencies

Keep the COBOL that works, understand it, and modernise on their own schedule.

Risk, compliance and audit

Readable rules, proofs, DMN and a tamper-evident review trail.

Change boards and developers

Impact of every change, with examples confirmed on both compiled versions.

Regulators

Explainable decisions: rule, COBOL lines, risk factors and a checked explanation.

Customers

Consistent, explained outcomes — and fewer hidden errors in their accounts.

Built with open source and IBM tools

COMPONENT	USE IN LEGACYLENS	LICENCE
IBM Z Open Editor samples (SAM1, SAM2)	real IBM COBOL to analyse and verify, bundled unmodified	Apache-2.0
GnuCOBOL 3.1.2	compiles and runs the COBOL on s390x — the reference for every check	GPL-3.0
z3 4.8.12	symbolic analysis, proofs, examples and change-impact regions	MIT
IBM Snap ML 1.17.2 + zDNN	ONNX import and inference on the z17 AI accelerator	IBM licence / Apache-2.0
scikit-learn, skl2onnx, ONNX	training, export, exact reference evaluation	BSD-3 / Apache-2.0
IBM Granite 3.3 2B + llama.cpp	explanations and rule documentation on s390x, big-endian GGUF	Apache-2.0 / MIT
OMG DMN 1.3 schemas	validating the exported decision models	OMG
FastAPI, uvicorn, pandas, NumPy	web service, UI and data handling	MIT / BSD
podman, systemd, Ubuntu 24.04	rootless container and services on LinuxONE	Apache-2.0 / various
UCI Default of Credit Card Clients	30,000 accounts for training, testing and the portfolio	CC BY 4.0

What we know, and where we go next

Limitations today

- No CICS/DB2 EXEC blocks, STRING/UNSTRING/INSPECT/SEARCH, SORT or indexed files yet — reported, never guessed.
- Agreement on tens of thousands of inputs is strong evidence, not a proof; z3 results are proofs within the encoding.
- DMN files are schema-valid and checked with our own FEEL evaluator, not yet a third-party engine.
- On this VM the CPU path beats the AI accelerator for these small tree models.
- Granite 2B wording can be awkward; the guard checks facts, not style.

Next steps

- CICS, DB2 and the remaining COBOL statements; z/OS datasets and JCL.
- Run the DMN exports in a third-party decision engine.
- Larger and deep-learning risk models and batch scoring, where the AI accelerator pays off.
- Change impact as a release gate in CI: block a release that moves decisions unexpectedly.
- A pilot on a real COBOL decision system with its owners.

THANK YOU

LegacyLens

COBOL you can prove.

Changes you can see before they ship.

Real-time AI you can trust.

Built and run on IBM LinuxONE · IBM z17

Team Aviator

Piyush Anand · Somesh Srivastava · Praman Patel · Paraj Prasad · Mohit Kumar A

github.com/GeekSomesh/LegacyLens

